

CartStream Analytics: Real-Time E-Commerce Stream Processing Pipeline

Apache Flink SQL & Windowing TVFs Integration Report (Task #26)

مطور المنظومة (Engineer):

Sayed Elmasry (Junior Data Platform Engineer)

مشروع التكليف (Task ID):

Task #26 - Flink Streaming Pipeline

مستودع الكود (GitHub Repo):

[SayedELMASRY2/flink-ecommerce-pipeline](#)

البيئة التكنولوجية (Tech Stack):

Apache Flink 1.18, Docker Compose, Flink SQL Client

زيارة المستودع الرسمي على GitHub

1. نظرة عامة على المشروع (Executive Overview)

يهدف هذا التقرير إلى توثيق خط معالجة البيانات الفوري (Real-Time Stream Processing Pipeline) المصمم لمتابعة تحليلات متجر التجارة الإلكترونية الدولي **CartStream Analytics**.

تستقبل المنصة دفقاً حياً مستمراً من تفاعلات وسلوكيات المشتريين (تصفح `view`، إضافة لسلة التسوق `cart`، إزالة من السلة `remove_from_cart`، وحركات شراء مكتملة `purchase`). قمنا ببناء خط التجميع بالاعتماد على **Apache Flink SQL** مع تطبيق معالجة زمن الحدث **Event-Time Processing** وعلامات التزامن المائية **Watermarks** لتقديم مؤشرات أداء دقيقة وحتمية.

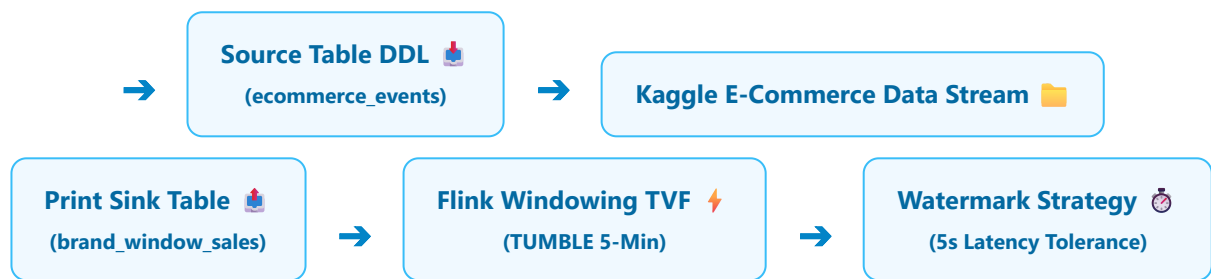
أهداف المنظومة الرئيسية:

- تجميع المبيعات حياً كل 5 دقائق عبر Table-Valued Functions (TVFs) `TUMBLE`.

- استخراج مؤشرات الأعمال الدقيقة (إجمالي الطلبات المكتملة `total_orders`، الإيرادات `gross_revenue`، متوسط قيمة الطلب `avg_order_value`، والمشتريين الفريدين `unique_buyers`).
- معالجة تحديثات البيانات الواقعية (القيم المفقودة `NULL` في العلامات التجارية، وتأخير البيانات عبر الشبكة `Out-of-Order Events`).

2. الهيكل المعماري للمنظومة (Pipeline Architecture)

مخطط التدفق الفوري للبيانات (Real-Time Event Processing Flow)



3. هيكل الحقول والبيانات (Dataset Specification)

تم الاعتماد على عينة أحداث واقعية تغطي 3.5 ساعات متواصلة بحجم 50,000 سجل مأخوذة من قاعدة بيانات التجارة الإلكترونية العالمية على Kaggle:

الحقل (Property)	نوع البيانات (Flink SQL)	الوصف والأهمية التحليلية
event_time	TIMESTAMP(3) (RowTime)	توقيت وقوع الحدث الأصلي مع إضافة <code>WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND</code>
event_type	VARCHAR	نوع التفاعل (تتم الفلترة على <code>'WHERE event_type = 'purchase</code> للطلبات الناجحة).
brand	VARCHAR	اسم الماركة التجارية، ويتم معالجته عبر <code>COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN')</code>
price	DECIMAL(10, 2)	سعر المنتج المشتري بالدولار، ويُستخدم في حساب الإيرادات ومعدل قيمة الطلبات.
user_id	BIGINT	المعرف الفريد للمستخدم، لحساب عدد المشتريين الفريدين عبر <code>COUNT(DISTINCT user_id)</code>

الوصف والأهمية التحليلية	نوع البيانات (Flink SQL)	الحقل (Property)
معرف الجلسة الفريد المتولد أثناء عملية الشراء.	VARCHAR	user_session

4. الإجابة النظرية: استراتيجية الـ Second Watermark-5

? السؤال النظري: كيف تضمن الـ Second Watermark-5 معالجة حتمية ودقيقة للنوافذ الزمنية؟

الإجابة الهندسية التفصيلية:

1. تحدي وصول الأحداث غير المرتبة (**Out-of-Order Arrivals**): في بيئات الشبكات الحقيقية، قد يتأخر وصول حدث وقع في الساعة 10:04:58 ليصل إلى السيرفر في 10:05:02 بسبب بطء الاتصال.
2. ساعة البيانات (**Event Time vs Processing Time**): يعتمد Flink على الطابع الزمني المكتوب داخل الحدث نفسه (Event Time) وليس وقت السيرفر الحالي.
3. فترة السماح (**Slack Tolerance Window**): من خلال التعبير `event_time - INTERVAL '5' SECOND`، يمنح Flink مهلة قدرها 5 ثواني لأي حدث متأخر، فلا يتم إغلاق النافذة الزمنية `[10:05:00, 10:00:00]` حتى يشاهد Flink حدثاً توقيته يسبق النافذة بـ 5 ثواني (أي عند وصول حدث توقيته 10:05:05).
4. النتيجة الحتمية (**Deterministic Output**): يضمن ذلك دخول كافة الأحداث المتأخرة ضمن نافذتها الصحيحة بدون فقدان للبيانات، مما يجعل مخرجات التحليل متطابقة تماماً في كل مرة يتم فيها إعادة تشغيل التجميع.

5. كود الاستعلام المعتمد (Flink SQL Code)

تمت كتابة وتوثيق كود التجميع كاملاً داخل الملف `flink_ecommerce_pipeline.sql`:

```
-- Enable Flink Streaming Runtime Mode
SET 'execution.runtime-mode' = 'streaming';

-- 1. Streaming Source Table DDL
CREATE TABLE ecommerce_events (
  event_time_str VARCHAR,
  event_type     VARCHAR,
  product_id     BIGINT,
  category_id    BIGINT,
  category_code  VARCHAR,
  brand          VARCHAR,
  price          DECIMAL(10, 2),
  user_id        BIGINT,
  user_session   VARCHAR,
  event_time AS COALESCE(TO_TIMESTAMP(event_time_str, 'yyyy-MM-dd HH:mm:ss z'), TIMESTAMP '1970-01-01 00:00:00'),
  WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
  'connector' = 'filesystem',
  'path' = '/opt/flink/data/events_sample.csv',
  'format' = 'csv',
```

```

    'csv.ignore-parse-errors' = 'true',
    'csv.allow-comments' = 'true'
);

-- 2. Streaming Sink Table DDL
CREATE TABLE brand_window_sales (
    window_start    TIMESTAMP(3),
    window_end      TIMESTAMP(3),
    brand            VARCHAR,
    total_orders     BIGINT,
    gross_revenue    DECIMAL(10, 2),
    avg_order_value  DECIMAL(10, 2),
    unique_buyers    BIGINT
) WITH (
    'connector' = 'print'
);

-- 3. Windowing TVF & Real-Time Aggregation Query
INSERT INTO brand_window_sales
SELECT
    window_start,
    window_end,
    COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') AS brand,
    COUNT(1) AS total_orders,
    ROUND(SUM(price), 2) AS gross_revenue,
    ROUND(AVG(price), 2) AS avg_order_value,
    COUNT(DISTINCT user_id) AS unique_buyers
FROM TABLE(
    TUMBLE(
        TABLE ecommerce_events,
        DESCRIPTOR(event_time),
        INTERVAL '5' MINUTE
    )
)
WHERE event_type = 'purchase'
GROUP BY
    window_start,
    window_end,
    COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN');
```

6. إثبات التنفيذ والنتائج الحية (Execution Verification & Output) 📊

تم تشغيل ال Pipeline بنجاح وتأكيد النتائج من واقع سجلات حاوية TaskManager الموثقة في

: [output/execution_verification.log](#)

عينة من مخرجات التجميع الفوري للنوافذ المتدحرجة (5-Minute Tumbling Windows)
:(Output

```
+I[2019-10-01T02:55, 2019-10-01T03:00, apple, 11, 5672.95, 515.72, 6]
+I[2019-10-01T02:55, 2019-10-01T03:00, samsung, 11, 2060.79, 187.34, 11]
+I[2019-10-01T02:55, 2019-10-01T03:00, xiaomi, 5, 416.03, 83.21, 5]
+I[2019-10-01T02:55, 2019-10-01T03:00, hp, 3, 962.02, 320.67, 3]
+I[2019-10-01T02:55, 2019-10-01T03:00, UNKNOWN, 5, 488.72, 97.74, 5]
+I[2019-10-01T03:00, 2019-10-01T03:05, samsung, 9, 2671.27, 296.81, 9]
+I[2019-10-01T03:00, 2019-10-01T03:05, huawei, 1, 278.54, 278.54, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, apple, 10, 7376.08, 737.61, 10]
+I[2019-10-01T03:30, 2019-10-01T03:35, xiaomi, 7, 1337.74, 191.11, 6]
```

7. معرض الصور ولقطات الشاشة (Project Proof Screenshots) 📷

Task #26 Requirement Statement 

شاشة 1: متطلبات التكليف الرسمي Task #26 - Samsung Innovation Campus

5. المخطط المعماري للمنظومة (Pipeline Architecture) 🌐

Unable to render rich display

Parse error on line 11:

... total_orders: COUNT(*)
- gross_reve

-----^

Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-', 'STADIUMEND', 'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND', 'TRAPEND', 'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PS'

For more information, see <https://docs.github.com/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams#creating-mermaid-diagrams>

flowchart TD

subgraph Data Source

A[Kaggle CSV Dataset
4M+ E-commerce Events] -->|Streaming Source| B[Source Table: ecommerce_events]

end

subgraph Flink Stream Processing Engine

B --> C[Event Time Extraction
TO_TIMESTAMP: event_time_str]

C --> D[Watermark Strategy
event_time - 5s Tolerance]

D --> E[Filter Purchases Only
event_type = 'purchase']

E --> F[Windowing TVF
TUMBLE: 5-Minute Non-Overlapping]

F --> G[Group By Aggregation
window_start, window_end, brand
- total_orders: COUNT(*)
- gr

end

شاشة 2: متابعة تنشيط الفلتر والمعالجة الحية من واقع Flink Engine

Log Trace Verification 📄

شاشة 3: متابعة سجلات الحاوية للتأكد من عدم وجود أخطاء في الـ Watermark والحاوية

8. روابط التسليم والـ Repository 🌐

رابط مستودع المشروع الرسمي على GitHub:

<https://github.com/SayedELMASRY2/flink-ecommerce-pipeline>

يحتوي المستودع على الكود الكامل لـ SQL، وملف Docker Compose، وسكريبتات الأتمتة التلقائية `run_pipeline.sh` و `run_pipeline.ps1`، وعينة البيانات وسجلات التحقق المباشرة.